

GRU#

[https://pytorch.org/docs/stable/generated/torch.nn.quantized.dynamic.GRU.](https://pytorch.org/docs/stable/generated/torch.nn.quantized.dynamic.GRU.html)

Description:

Applies a multi-layer gated recurrent unit (GRU) RNN to an input sequence. For each element in the input sequence, each layer computes the following function: where h_t is the hidden state at time t , x_t is the input at time t , h_{t-1} is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and r_t , z_t , n_t are the reset, update, and new gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product. In a multilayer GRU, the input $x^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h^{(l-1)}$ of the previous layer multiplied by dropout $\delta^{(l-1)}$ where each $\delta^{(l-1)}$ is a Bernoulli random variable which is 0 with probability dropout.
input_size – The number of expected features in the input x

Function Signature

```
class torch.nn.quantized.dynamic.GRU(*args, **kwargs)
[source]#
```

Parameters

Inputs: input, h_0

Outputs: output, h_n

Shape:

Code Examples

```
>>> rnn = nn.GRU(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> output, hn = rnn(input, h0)
```

Notes & Warnings

NOTE All the weights and biases are initialized from $U(-k, k) \sqrt{k}$, $U(-k, k) \sqrt{k}$ where $k = \frac{1}{\text{hidden_size}}$

NOTE The calculation of new gate n_t subtly differs from the original paper and other frameworks. In the original implementation, the Hadamard product $(\odot)$ between r_t and the previous hidden state h_{t-1} is done before the multiplication with the weight matrix W and addition of bias:

$$n_t = \tanh(W_{in} x_t + b_{in} + W_{hn} (r_t \odot h_{t-1}) + b_{hn})$$

This is in contrast to PyTorch implementation, which is done after $W_{hn} h_{t-1}$:

$$n_t = \tanh(W_{in} x_t + b_{in} + r_t \odot (W_{hn} h_{t-1} + b_{hn}))$$

NOTE If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.

Detailed Documentation

Documentation

Applies a multi-layer gated recurrent unit (GRU) RNN to an input sequence.

For each element in the input sequence, each layer computes the following function:

where h_t is the hidden state at time t , x_t is the input at time t , h_{t-1} is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and r_t , z_t , n_t are the reset, update, and new gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer GRU, the input $x^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h^{(l-1)}$ of the previous layer multiplied by dropout $\delta^{(l-1)}$ where each $\delta^{(l-1)}$ is a Bernoulli random variable which is 0 with probability dropout.

input_size – The number of expected features in the input x

hidden_size – The number of features in the hidden state h

num_layers – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two GRUs together to form a stacked GRU, with the second GRU taking in outputs of the first GRU and computing the final results. Default: 1

bias – If False, then the layer does not use bias weights b_{ih} and b_{hh} . Default: True

batch_first – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False

dropout – If non-zero, introduces a Dropout layer on the outputs of each GRU layer except the last layer, with dropout probability equal to dropout. Default: 0

bidirectional – If True, becomes a bidirectional GRU. Default: False

input of shape (seq_len, batch, input_size): tensor containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` for details.

h_0 of shape (num_layers * num_directions, batch, hidden_size): tensor containing the initial hidden state for each element in the batch. Defaults to zero if not provided. If the RNN is bidirectional, num_directions should be 2, else it should be 1.

output of shape (seq_len, batch, num_directions * hidden_size): tensor containing the output features h_t from the last layer of the GRU, for each t . If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence. For the unpacked case, the directions can be separated using `output.view(seq_len, batch, num_directions, hidden_size)`, with forward and backward being direction 0 and 1 respectively.

Similarly, the directions can be separated in the packed case.

h_n of shape (num_layers * num_directions, batch, hidden_size): tensor containing the hidden state for $t = \text{seq_len}$

Like output, the layers can be separated using `h_n.view(num_layers, num_directions, batch, hidden_size)`.

Input1: (L, N, H_{in}) tensor containing input features where $H_{in} = \text{input_size}$ and L represents a sequence length.

Input2: (S,N,Hout)(S, N, H_{out})(S,N,Hout) tensor containing the initial hidden state for each element in the batch. Hout=hidden_sizeH_{out}=\text{hidden_size}Hout=hidden_size Defaults to zero if not provided. where S=num_layers*num_directionsS=\text{num_layers} * \text{num_directions}S=num_layers*num_directions If the RNN is bidirectional, num_directions should be 2, else it should be 1.

Output1: (L,N,Hall)(L, N, H_{all})(L,N,Hall) where Hall=num_directions*hidden_sizeH_{all}=\text{num_directions} * \text{hidden_size}Hall=num_directions*hidden_size

Output2: (S,N,Hout)(S, N, H_{out})(S,N,Hout) tensor containing the next hidden state for each element in the batch

weight_ih_l[k] – the learnable input-hidden weights of the kth\text{k}^{th}kth layer (W_ir|W_iz|W_in), of shape (3*hidden_size, input_size) for k = 0. Otherwise, the shape is (3*hidden_size, num_directions * hidden_size)

weight_hh_l[k] – the learnable hidden-hidden weights of the kth\text{k}^{th}kth layer (W_hr|W_hz|W_hn), of shape (3*hidden_size, hidden_size)

bias_ih_l[k] – the learnable input-hidden bias of the kth\text{k}^{th}kth layer (b_ir|b_iz|b_in), of shape (3*hidden_size)

bias_hh_l[k] – the learnable hidden-hidden bias of the kth\text{k}^{th}kth layer (b_hr|b_hz|b_hn), of shape (3*hidden_size)

All the weights and biases are initialized from $U(-k,k)\backslash\mathrm{mathcal{U}}(-\sqrt{k},\sqrt{k})U(-k,k)$ where $k=\frac{1}{\text{hidden_size}}$ k=hidden_size1

The calculation of new gate ntn_tnt subtly differs from the original paper and other frameworks. In the original implementation, the Hadamard product $(\odot)(\odot)$

between `rtr_trt` and the previous hidden state $h_{(t-1)}$ is done before the multiplication with the weight matrix W and addition of bias:

This is in contrast to PyTorch implementation, which is done after $W_{hn}h_{(t-1)}$

This implementation differs on purpose for efficiency.

If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype `torch.float16` 4) V100 GPU is used, 5) input data is not in `PackedSequence` format persistent algorithm can be selected to improve performance.